iscte
INSTITUTO UNIVERSITÁRIO DE LISBOA

emprego digital

Módulo 1: Programação em Linguagem Java

Aula 28
Serialização, Genericidade e Lambdas

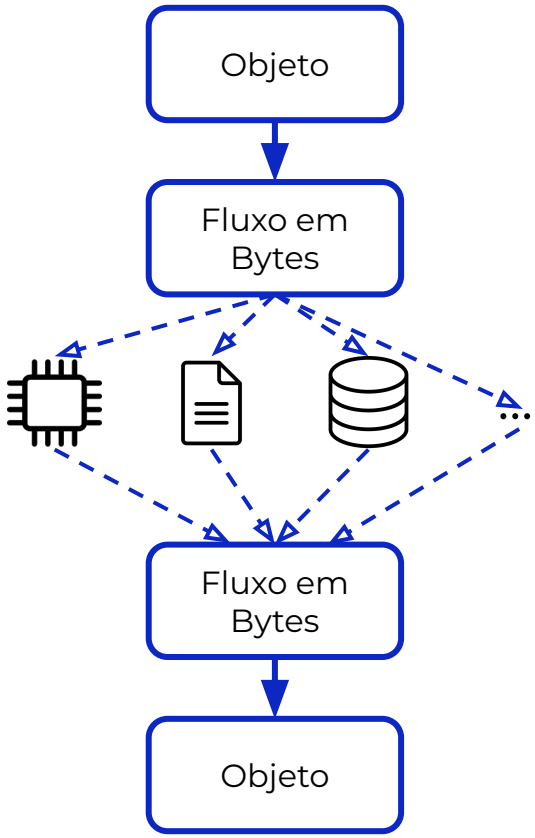
UPskill

Serialização

iscte
INSTITUTO
UNIVERSITÁRIO
DE LISBOA
emprego
digital

Serialização

- Mecanismo que permite transformar objetos, primitivos e dados em bytes que podem ser guardados em ficheiros, enviados pela internet, etc.
- Objetos depois de serializados podem ser deserializados para serem transformados de volta no objeto
- No Java é possível serializar/deserializar mantendo o tipo e dados de forma coerente, mesmo entre vários dispositivos.



iscte
INSTITUTO UNIVERSITÁRIO DE LISBOA
emprego digital

Como Serializar Objeto?

```
public class Map implements Serializable {
    private List<ImageTile> tiles = new ArrayList<>();
    private Hero hero;
    private int level;

    public Map(Hero hero, int level) {
        this.hero = hero;
        readMapFile(level);
    }

    private List<ImageTile> readMapFile(int level) {
        //etc..
```

Implementar a interface *Serializable*

Todos os atributos também têm de ser *Serializable*

Primitivos do Java e classes da Java Collections já são *Serializable* por defeito

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA

emprego digital

Exemplo: **Serialização**

```
Hero hero = new Hero(new Position(3, 5));
Map map = new Map(hero, 0);
try {
   FileOutputStream fileOut = new FileOutputStream("save.dat");
   ObjectOutputStream out = new ObjectOutputStream(fileOut);
   out.writeObject(map);
   out.close();
   fileOut.close();
} catch (IOException e) {
   System.out.println("Erro a salvar o mapa no ficheiro!");
}
```

Usamos o FileOutputStream para salvar um fluxo de dados num ficheiro

Usamos o ObjectOutputStream para transformar o objeto serializável num fluxo de dados

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA
emprego digital

Exemplo: **Deserialização**

```
try {
   FileInputStream fileIn = new FileInputStream("save.dat");
   ObjectInputStream in = new ObjectInputStream(fileIn);
   Map loadedMap = (Map) in.readObject();
   in.close();
   fileIn.close();
   System.out.println("Vida do hero: " + loadedMap.getHero().getHealth());
} catch (IOException e) {
   System.out.println("Erro a ler o ficheiro com o save do mapa!");
} catch (ClassNotFoundException e) {
   System.out.println("Não foi possível converter o objeto salvo no mapa!");
}
```

Para ler o fluxo de dados do ficheiro usamos o FileInputStream e ObjectInputStream

É necessário fazer *cast* do objeto lido para o Java saber como interpretar os dados

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA
emprego digital

Exercício 1

Crie um menu na consola que pede ao utilizador para introduzir "**1**" para ***Salvar Dados*** e "**2**" para ***Carregar Dados.***

Na opção ***Salvar Dados***, peça ao utilizador para introduzir o seu nome, uma nota (campo de texto livre) e um título. Por fim, peça ao utilizador o nome do ficheiro a guardar e salve esses dados num ficheiro.

Na opção ***Carregar Dados***, peça ao utilizador para introduzir o nome do ficheiro a carregar e mostre na consola a nota guardada.

Genericidade

Genericidade

- Adicionada no Java 5 como forma de providenciar verificações de tipos de dados a nível do compilador.
- Permite trabalhar com tipos de dados diferentes dentro das mesmas classes
- *Java Collections Framework* usa genericidade para criar as ***Lists***, ***Sets***, ***Maps***, etc
- Interface Comparable também usa genericidade para criar o método de interface para comparar um objeto com o outro do mesmo tipo.

Genericidade

```
List<Integer> list = new ArrayList<Integer>();
list.add(10);
list.add("10");

//...

List list = new ArrayList();
list.add(10);
list.add("10");

Integer num = (Integer) list.get(0);
Integer num2 = (Integer) list.get(1);
```

Instanciar a classe ArrayList como uma ArrayList **de** Integers

Compilador queixa-se que é uma lista de inteiros e não podemos adicionar uma String.

Instanciar sem definir o tipo genérico

Compilador permite adicionar qualquer tipo de objeto

Ao instanciar sem definir o tipo genérico, ao ir buscar o valor à lista é necessário fazer cast, e caso o objeto não seja do tipo certo, lança uma exceção.

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA emprego digital

Genericidade: **Comparable**

```
public interface Comparable<T> {
   public int compareTo(T o);
}
```

Utiliza-se o T (ou qualquer outro caracter) dentro dos símbolos <> para especificar que é do tipo genérico

Interface *Comparable* contém o método compareTo genérico que utiliza o T

```
public class Estudante implements Comparable<Estudante> {
   @Override
   public int compareTo(Estudante outro) {
       return outro.numero - this.numero;
   }
   //...
}
```

Quando implementado, passamos o tipo de objeto que queremos usar como genérico para ficar automaticamente especificado no método *compareTo*

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA
emprego digital

Genericidade: **Limitação de Tipos**

- Ao criar uma classe ou método genérico, é possível limitar os tipos de objetos que são aceites:

```
public class Test<T extends ImageTile> {
    private T tile;
    ...
}
```

T só pode ser subclasse de ImageTile

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA emprego digital

Genericidade: **Exemplo Rogue**

```
public interface Interactible<T> {
   public void interact(T o);
}
```

Exemplo com o propósito de criar um objeto “interagível”, ou seja, com o qual o Hero pode interagir e utilizar o método interact

```
public class Sword implements Interactible<Hero> {
   @Override
   public int interact(Hero hero) {
       hero.addToInventory(this);
       //...
   }
   //...
}
```

Ao implementar, passamos o Hero como objeto que vai interagir com o nosso *Sword* e usamos o método interact para adicionar a sword ao inventório, remover do mapa, etc

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA
emprego digital

Genericidade: **Exemplo Rogue**

```
public interface Interactible<T> {
   public void interact(T o);
}
```

Exemplo com o propósito de criar um objeto “interagível”, ou seja, com o qual o Hero pode interagir e utilizar o método interact

```
public class Sword implements Interactible<Hero> {
   @Override
   public int interact(Hero hero) {
       hero.addToInventory(this);
       //...
   }
   //...
}
```

Ao implementar, passamos o Hero como objeto que vai interagir com o nosso *Sword* e usamos o método interact para adicionar a sword ao inventório, remover do mapa, etc

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA
emprego digital

Exercício 2

Crie uma classe genérica ***Pair*** que aceita quaisquer dois tipos de objetos:

```
public class Pair<K, V> { ... }
```

A classe deve conter um par ***Key-Value*** como atributos e deve ter um construtor, *getters* e *setters* apropriados.

Teste a classe criando uma instância do par (parametrize a chave e o valor) e experimente o construtor, os *getters* e *setters*.

Lambdas

iscte
INSTITUTO
UNIVERSITÁRIO
DE LISBOA
emprego
digital

Lambdas

- Uma expressão lambda é um bloco de código que recebe parâmetros e retorna um valor.
- São semelhantes a métodos, mas não precisam de um nome e podem ser implementados diretamente no código

```
parameter -> expression
(parameter1, parameter2) -> expression
```

- As expressões são limitadas e não podem conter variáveis, atribuições ou instruções como if e for. Para usar expressões mais complexas, o bloco de código pode conter chavetas nas extremidades.

```
(parameter1, parameter2) -> { code block }
```

iscte
INSTITUTO UNIVERSITÁRIO DE LISBOA
emprego digital

Como usar o **Lambda**

- Expressões lambda **podem ser armazenadas em variáveis** se o tipo de variável for uma interface que possui apenas um método.
- A expressão lambda deve ter o mesmo número de parâmetros e o mesmo tipo de retorno desse método.
- A interface Consumer é um exemplo, usada em listas.

```java
List<Integer> numbers = new ArrayList<>();
numbers.add(5);
numbers.add(9);
numbers.add(8);
numbers.add(1);
numbers.forEach(
     (n) -> { System.out.println(n);}
);
// alternativa
numbers.forEach(System.out::println);
```

```java
List<Integer> numbers = new ArrayList<>();
numbers.add(5);
numbers.add(9);
numbers.add(8);
numbers.add(1);
Consumer<Integer> method = (n) -> { System.out.println(n);};
numbers.forEach(method);
```

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA

emprego digital

Como usar o **Lambda**

```
interface StringFunction {
   String run(String str);
}

public class Main {
   public static void main(String[] args) {
       StringFunction exclaim = (s) -> s + "!";
       StringFunction ask = (s) -> s + "?";
       printFormatted("Hello", exclaim);
       printFormatted("Hello", ask);
   }
   public static void printFormatted(String str, StringFunction format){
       String result = format.run(str);
       System.out.println(result);
   }
}
```

Resultado na consola:

```
Hello!
Hello?
```

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA
emprego digital

Como usar o **Lambda**

- Comparador clássico:

```
Comparator<Pessoa> byName = new Comparator<Pessoa>() {
   @Override
   public int compare(Pessoa o1, Pessoa o2) {
       return o1.getName().compareTo(o2.getName());
   }
};
```

- Equivalente usando uma função lambda:

```
Comparator<Pessoa> byName =
       (Pessoa o1, Pessoa o2)->o1.getName().compareTo(o2.getName());
```

Exercício 3

Considere uma lista do tipo Integer. Crie um programa que imprima na consola o dobro de todos os elementos da lista usando expressões lambda.

Exercício 4

Considere uma lista do tipo Carro(String matricula, int ano). Faça um programa onde insere 10 carros e ordene a lista por matriculas e anos usando:

- Um comparador
- Uma expressão lambda

O futuro profissional começa aqui

iscte
INSTITUTO UNIVERSITÁRIO DE LISBOA

emprego digital

UPskill